

Experiment No: 1

Title: Write a Python Program to Solve the 8-Puzzle Problem

Aim

To develop a Python program to solve the 8-Puzzle Problem using the Breadth First Search (BFS) algorithm and obtain the sequence of moves from the initial state to the goal state.

Objective

To implement the Breadth First Search (BFS) algorithm for solving the 8-Puzzle Problem by exploring all possible states until the goal state is reached.

Algorithm

- **Step 1:** Define the goal state of the puzzle.
- **Step 2:** Read the initial state from the user.
- **Step 3:** Insert the initial state into the queue.
- **Step 4:** Remove the first state from the queue.
- **Step 5:** Generate all possible valid moves of the blank tile.
- **Step 6:** Compare each new state with the goal state.
- **Step 7:** If the goal state is found, display the solution.
- **Step 8:** Otherwise, add unvisited states to the queue and repeat the process until the goal state is reached.

Program

```
from collections import deque
```

```
# Goal State
```

```
goal = [1, 2, 3,  
        4, 5, 6,  
        7, 8, 0]
```

```
# Print the puzzle as a 3x3 matrix
```

```
def print_board(state):  
    for i in range(0, 9, 3):  
        print(state[i], state[i+1], state[i+2])
```

```
print()
```

```
# Generate all possible moves
```

```
def get_moves(state):
```

```
    moves = []
```

```
    pos = state.index(0)
```

```
    # Up
```

```
    if pos >= 3:
```

```
        new = state[:]
```

```
        new[pos], new[pos-3] = new[pos-3], new[pos]
```

```
        moves.append(new)
```

```
    # Down
```

```
    if pos <= 5:
```

```
        new = state[:]
```

```
        new[pos], new[pos+3] = new[pos+3], new[pos]
```

```
        moves.append(new)
```

```
    # Left
```

```
    if pos % 3 != 0:
```

```
        new = state[:]
```

```
        new[pos], new[pos-1] = new[pos-1], new[pos]
```

```
        moves.append(new)
```

```
    # Right
```

```
    if pos % 3 != 2:
```

```
        new = state[:]
```

```
        new[pos], new[pos+1] = new[pos+1], new[pos]
```

```
moves.append(new)
```

```
return moves
```

```
# BFS Algorithm
```

```
def bfs(start):
```

```
    queue = deque([start])
```

```
    visited = []
```

```
    while queue:
```

```
        state = queue.popleft()
```

```
        if state == goal:
```

```
            print("Goal State Reached!\n")
```

```
            print_board(state)
```

```
            return
```

```
        if state not in visited:
```

```
            visited.append(state)
```

```
            print("Current State:")
```

```
            print_board(state)
```

```
            for move in get_moves(state):
```

```
                if move not in visited:
```

```
                    queue.append(move)
```

```
print("No Solution Found")
```

```
# User Input
print("Enter the Initial State (Use 0 for Blank):")
start = list(map(int, input().split()))

bfs(start)
```

Sample Input

Enter the Initial State (Use 0 for Blank):

1 2 3 4 0 6 7 5 8

Sample Output

Enter the Initial State (Use 0 for Blank):

1 2 3 4 0 6 7 5 8

Current State:

1 2 3

4 0 6

7 5 8

Current State:

1 0 3

4 2 6

7 5 8

Current State:

1 2 3

4 5 6

7 0 8

Current State:

1 2 3

0 4 6

7 5 8

Current State:

1 2 3

4 6 0

7 5 8

Current State:

0 1 3

4 2 6

7 5 8

Current State:

1 3 0

4 2 6

7 5 8

Current State:

1 2 3

4 5 6

0 7 8

Goal State Reached!

1 2 3

4 5 6

7 8 0

Result

Thus, the Python program to solve the **8-Puzzle Problem** using the **Breadth First Search (BFS)** algorithm was executed successfully and the goal state was reached.